Correspondências entre UML, código Java e estado de objetos: um instrumento experimental e sua verificação técnica

Versão preliminar 0.1, 18 de outubro de 2026. Autoria, afiliação e veículo a confirmar pelo autor. Não submetido. A avaliação educacional não foi realizada.

Resumo

A relação entre a estrutura de uma classe e o comportamento de suas instâncias constitui uma questão relevante no ensino introdutório de orientação a objetos. Este trabalho apresenta um instrumento experimental de escopo restrito para investigar correspondências explícitas entre um cartão UML, regiões do código Java e estados de objetos nas fronteiras de chamadas de métodos. O demonstrador utiliza objetos reais na JVM e oferece duas condições com executor compartilhado: uma com destaques e mensagens de correspondência e outra sem esse apoio adicional. A implementação foi verificada por compilação, testes determinísticos e renderização da interface. Na execução inicial preservada, 28 verificações Java e cinco testes do gerador passaram. Esses resultados sustentam a funcionalidade dos cenários exercitados, mas não demonstram aprendizagem, usabilidade ou demanda comercial. São apresentados um protocolo educacional prospectivo e os limites necessários à interpretação das evidências. A contribuição candidata é um instrumento controlável e rastreável, sem reivindicar a primeira integração entre UML e execução.

Palavras-chave: orientação a objetos; Java; UML; visualização de programas; instrumento de pesquisa.

1. Introdução

Representar um programa exige distinguir estruturas declaradas, objetos criados e valores que mudam durante sua execução. Em uma atividade de orientação a objetos, observar dois nomes de referência não basta para decidir se existem dois objetos ou se ambos designam a mesma instância. O problema investigado é tornar essas relações observáveis e avaliáveis em uma experiência delimitada, sem presumir que a representação visual produza compreensão por si.

Kölling et al. (2003) apresentam o BlueJ como ambiente pedagógico com interação direta entre classes e objetos. Esse antecedente impede tratar a simples combinação de diagrama e execução como novidade. O presente trabalho concentra-se no desenho de condições comparáveis e na rastreabilidade entre mecanismo implementado, verificações técnicas e uma avaliação educacional futura.

A pergunta educacional é: qual é o efeito adicional de correspondências explícitas entre UML, fonte e estado do receptor sobre a compreensão de classes, instâncias e mudanças de estado? Este manuscrito ainda não responde empiricamente a essa pergunta. Seu objetivo realizado é apresentar o demonstrador e verificar comportamentos técnicos específicos; o objetivo educacional permanece prospectivo.

2. Fundamentação e trabalhos relacionados

O framework DeFT propõe considerar o desenho, as funções e as tarefas cognitivas associadas ao uso de múltiplas representações (Ainsworth, 2006). Sua aplicação aqui é uma decisão de projeto: tornar explícitas relações entre as representações e avaliar a compreensão resultante. Não se deduz desse framework uma garantia de ganho educacional para esta interface.

Hundhausen, Douglas e Stasko (2002) sintetizam estudos de visualização de algoritmos e destacam a relevância da maneira como a tecnologia é utilizada. Por isso, o protocolo inclui prever, executar e explicar, em ambas as condições. A transferência dessa fundamentação para conceitos de objetos precisa ser examinada no estudo próprio.

Também existem antecedentes de visualização dinâmica. Jeliot 3 aborda dados e controle de execução (Moreno et al., 2004), enquanto JIVE representa estruturas de objetos e histórico de chamadas (Gestwicki e Jayaraman, 2004). A comparação desta pesquisa é documental, não um ensaio instalado de desempenho ou usabilidade dessas ferramentas.

Assim, a contribuição candidata não é uma categoria inédita de software. É um instrumento pequeno, com executor compartilhado e intervenção visual delimitada, acompanhado de materiais para examinar sua utilidade e seus limites. A relevância dessa contribuição deverá ser julgada frente aos antecedentes e ao estudo futuro.

3. Método de desenvolvimento e verificação

O trabalho organiza a construção e a avaliação técnica de um artefato em uma perspectiva de Design Science Research, conforme a articulação metodológica discutida por Hevner et al. (2004). A existência do artefato e a passagem em testes não são tratadas como conclusão de todo o ciclo de investigação educacional.

Primeiro, foram identificados antecedentes e delimitado um mecanismo. Em seguida, foram implementados um exemplo Java, um executor de chamadas e uma interface experimental. A verificação técnica utilizou cenários determinísticos com valores conhecidos e nenhuma pessoa participante. Os fontes, o commit avaliado, os hashes e os logs foram preservados em repositório público.

A busca de literatura foi exploratória, com conferência em editoras e repositórios institucionais. Foram consultadas seções de textos originais e, para algumas fontes, apenas resumos e metadados. Não se declara revisão sistemática, leitura integral de todo o corpus ou auditoria bibliográfica exaustiva. Essas limitações estão registradas na matriz de evidências.

4. Instrumento e mecanismo

O demonstrador implementa BankAccount, com um saldo decimal e métodos para depositar, retirar e consultar. Referências distintas podem designar instâncias independentes ou a mesma instância por alias. Cada operação válida utiliza métodos reais da classe; o executor registra snapshots imutáveis antes e depois da chamada.

A interface apresenta um cartão estrutural UML, o fonte compilado e os objetos/estados. Marcadores explícitos delimitam regiões de fonte. Na condição linked, selecionar ou executar um membro destaca a região correspondente e evidencia referências alteradas.

Na condição plain, o executor, os dados e as atividades permanecem iguais, sem esses destaques ou mensagens adicionais. As regiões representam métodos, não instruções individuais rastreadas pela JVM.

O registro local de previsões e explicações conserva a quantidade de eventos existente no momento de cada resposta. Isso possibilita examinar a sequência, mas não impede registros tardios. O pesquisador deve definir a regra de aderência antes da análise. A exportação é local e não transmite dados pela rede.

Um gerador separado produz esqueletos Java a partir de um modelo estrutural restrito. Os corpos gerados exigem implementação; o exemplo executável tem corpos definidos explicitamente. Não há editor UML geral, sincronização bidirecional, suporte a Java arbitrário ou transformação automática de lógica de negócio descrita em diagramas.

5. Resultados técnicos observados

A execução inicial utilizou JDK Temurin 17.0.20.1 em runner Linux. O esqueleto gerado e o exemplo implementado foram compilados separadamente. Passaram 28 verificações Java e cinco testes Python do gerador. Os arquivos originais correspondem ao commit 5cf946e5a77439c3c41759536c006d6d8a2e3810 e à execução 39083293325 do GitHub Actions.

Dimensão	Evidência observada	Conclusão permitida
Instâncias independentes	Depósito em uma conta não modificou a outra	Correção do cenário exercitado
Alias	Duas referências com mesma identidade observaram a mudança	Correção do cenário de alias
Estado e operações	Snapshots, decimais e rejeições foram conferidos	Comportamentos testados consistentes
Comparação A/B	Estados e traces iguais; destaque diferente	Executor compartilhado nos cenários
Respostas	Contagens de eventos antes/depois preservadas	Registro técnico de cronologia
Interface	Duas condições renderizadas fora da tela	Renderização técnica, não usabilidade

Esses testes não produzem estimativa de efeito educacional. A quantidade de verificações não é tamanho amostral, e seus resultados não podem ser interpretados como respostas de estudantes. Em uma execução posterior, nove verificações da interface gráfica passaram em display virtual: janela visível, criação e depósito por botões, preservação da outra instância, exibição legível de estado e membro UML, registro de respostas, exportação pelo seletor de arquivos e conteúdo exportado. Essa evidência corresponde ao commit 89a84aa3132eb6c369a2dfe7ff2057716fe4b0df e à execução 39104951676. Não houve avaliação humana de usabilidade ou aprendizagem.

6. Avaliação educacional prospectiva

O protocolo propõe comparação entre linked e plain, com conteúdo, instrução e tempo equivalentes. O desfecho primário candidato é um escore conceitual de transferência próxima, ajustado ao conhecimento inicial. Os itens e a rubrica foram preparados, mas ainda requerem revisão e piloto. Não foram recrutados estudantes nem aplicados questionários.

Instituição, disciplina, quantidade acessível, amostra, alocação e procedimentos institucionais pertinentes não estão confirmados. Por isso, não se declara um protocolo principal congelado. Se o contexto permitir apenas uma investigação sem comparação adequada, será necessário rever o alcance da pergunta antes da coleta.

Uso, utilidade percebida e disposição a pagar deverão ser investigados separadamente. Nenhum desses resultados pode ser derivado de testes de software. A experiência comercial ainda é uma hipótese de produto, sem vendas ou demanda demonstradas.

7. Discussão e ameaças à validade

Os resultados técnicos permitem inspecionar a implementação do mecanismo, porém não estabelecem sua relevância educacional. O controle de um executor comum reduz uma fonte de diferença entre as condições; outras diferenças, como atenção ou familiaridade com destaques, permanecem possíveis explicações de resultados futuros.

O escopo de uma classe favorece rastreabilidade e reduz complexidade, mas limita a generalização para herança, coleções, concorrência e programas maiores. O mapeamento depende de marcadores e pode falhar se o fonte for alterado de forma incompatível. A interface desktop não foi validada em Android, na versão web existente ou com tecnologias assistivas.

A literatura selecionada inclui trabalhos fundacionais e descrições de ferramentas. A análise documental não demonstra ausência de recursos em sistemas concorrentes. A leitura integral e a verificação completa de correções/retratações ainda são pendências. A proposta necessita revisão humana e contextual antes de uma submissão que alegue eficácia educacional.

8. Conclusão

Foi desenvolvido um demonstrador com duas condições visuais e execução real de objetos Java. A verificação inicial confirmou os cenários técnicos especificados, com evidências e limites preservados. O resultado alcançado é um instrumento para pesquisa e sua validação técnica restrita; a hipótese de melhoria de compreensão permanece sem teste com participantes.

O próximo avanço científico depende de contexto, revisão dos instrumentos e estudo educacional apropriado. Não se afirma novidade ampla, aprovação acadêmica, aceite editorial ou viabilidade comercial. A conclusão dessas etapas exigirá suas evidências próprias.

Disponibilidade e declarações

Fontes e materiais: https://github.com/aloisiocosta-professor/trama-ide-web/tree/docs/research-bootstrap . Protocolo original preservado; versões complementares separadas. O repositório público não substitui uma licença de reutilização, ainda a definir pelo autor. Este texto foi preparado com assistência de IA; revisão humana, autoria e responsabilidade final não foram declaradas como concluídas. Nenhuma participação humana na avaliação técnica foi reportada.

Referências

AINSWORTH, S. DeFT: A conceptual framework for considering learning with multiple representations. Learning and Instruction, v. 16, n. 3, p. 183-198, 2006. DOI: 10.1016/j.learninstruc.2006.03.001.

GESTWICKI, P. V.; JAYARAMAN, B. JIVE: Java interactive visualization environment. OOPSLA '04 Conference Companion, p. 226-227, 2004. DOI: 10.1145/1028664.1028762. O registro Crossref usa o título curto JIVE; o título expandido consta do registro institucional.

HEVNER, A. R.; MARCH, S. T.; PARK, J.; RAM, S. Design Science in Information Systems Research. MIS Quarterly, v. 28, n. 1, 2004. DOI: 10.2307/25148625.

HUNDHAUSEN, C. D.; DOUGLAS, S. A.; STASKO, J. T. A Meta-Study of Algorithm Visualization Effectiveness. Journal of Visual Languages & Computing, v. 13, n. 3, p. 259-290, 2002. DOI: 10.1006/jvlc.2002.0237.

KÖLLING, M.; QUIG, B.; PATTERSON, A.; ROSENBERG, J. The BlueJ system and its pedagogy. Computer Science Education, v. 13, n. 4, p. 249-268, 2003. DOI: 10.1076/csed.13.4.249.17496.

MORENO, A.; MYLLER, N.; SUTINEN, E.; BEN-ARI, M. Visualizing programs with Jeliot 3. Proceedings of the Working Conference on Advanced Visual Interfaces, p. 373-376, 2004. DOI: 10.1145/989863.989928.